Il ciclo WHILE: lancia il dado finché…

2INF · lezione del 20/10/2026

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

Che cosa abbiamo fatto

1. **Quando:** 20/10/2026, 2 ore, Laboratorio.
2. **Argomenti:** Ciclo WHILE in Lazarus («ripeti finché»): teoria, ripasso, gioco del dado (lancia finché esce il numero magico e conta i tiri), ciclo infinito e come fermarlo; verifica.

La teoria e l'esercitazione

Le schede della lezione

1. Dal FOR al WHILE: quando non sai quante volte

Con il **for** sai prima quanti giri fai: `for x := 1 to 10 do` fa 10 giri, sempre.
Ma se lanci un dado finché esce il 6, **quante volte lo lanci?** Non lo sa nessuno: magari 1, magari 15.
Per questo c'è il **while**: «**finché** la condizione è vera, ripeti». Si ferma quando la condizione diventa falsa.
Nella vita: *finché* ho fame, mangio; *finché* il semaforo è rosso, aspetto.

2. Come si scrive

```
while condizione do begin   istruzioni da ripetere; end;
```

Si legge: **finché** la condizione è vera, **fai** quello che c'è tra begin ed end.
La condizione si controlla **prima** di ogni giro. Se è falsa già all'inizio, il while non fa **nessun** giro.
Le condizioni sono quelle dell'IF: `=` uguale, `<>` diverso, `<` minore, `>` maggiore, `<=`, `>=`.
Esempio: conta da 1 a 5 con il while:

```
x := 1; while x <= 5 do begin   Memo1.Lines.Add(IntToStr(x));   x
```

Nel for il contatore aumenta da solo; nel while **lo fai aumentare tu** (`x := x + 1`).

3. Il pericolo: il ciclo infinito

Se dentro il while niente cambia la condizione, il ciclo **non finisce mai**:

```
x := 1; while x <= 5 do begin   Memo1.Lines.Add(IntToStr(x)); end
```

Qui manca `x := x + 1`: x resta 1 per sempre e il programma si **blocca**.
Se ti succede: in Lazarus menu **Esegui ⟶ Ferma** (oppure **Ctrl+F2**). Succede a tutti i programmatori, anche ai pro: si ferma e si corregge.
Regola: dentro il while deve sempre esserci qualcosa che **prima o poi** rende falsa la condizione.

4. Il dado: Random

Per tirare un dado il computer usa i **numeri a caso**:

```
Randomize; dado := Random(6) + 1;
```

Randomize «mescola» i numeri: si scrive una volta all'inizio, altrimenti esce sempre la stessa sequenza.
Random(6) dà un numero da **0 a 5**; con **+ 1** diventa da **1 a 6**, come un dado vero.
Per una moneta: `Random(2)` dà 0 o 1 (testa o croce).

5. Il programma: lancia finché esce il numero magico

L'interfaccia: in alto **Numero magico (1-6)** [6], al centro un Memo grande, in basso un bottone **Lancia!**. Il codice del bottone, con i rientri giusti:

```
var dado, magico, tiri: integer; begin   Randomize;   magico := S
```

Leggilo: prendo il numero magico dalla casella; tiri parte da 0; **finché** il dado è **diverso** dal numero magico, tiro il dado, conto il tiro e lo scrivo nel Memo. Quando esce, scrivo dopo quanti tiri.
Perché `dado := 0` prima del while? Perché all'inizio il dado non è ancora uscito: 0 è diverso da qualsiasi numero magico, quindi il primo giro si fa sempre.

6. LA PRATICA, passi 1-5: il form

Un passo alla volta.
1) Apri Lazarus: **Progetto ⟶ Nuovo progetto ⟶ Applicazione ⟶ Ok**.
2) Metti un **TLabel** con Caption **Numero magico (1-6)**.
3) Accanto un **TEdit** (Edit1): nella proprietà **Text** scrivi **6**.
4) Al centro un **TMemo** grande con **ScrollBars = ssBoth**; cancella le righe di **Lines**.
5) In basso un **TButton** con Caption **Lancia!**.

7. LA PRATICA, passi 6-9: il codice e la prova

6) **Doppio clic** sul bottone: si apre `procedure TForm1.Button1Click`.
7) Scrivi il codice della scheda 5, riga per riga, con i rientri di 2 spazi. La riga `var dado, magico, tiri: integer;` va **sopra** il begin.
8) Premi **F9**.
9) Premi **Lancia!** più volte: ogni volta i tiri sono diversi. **FATTO!**
Prova: scrivi **9** nella casella e premi Lancia!. Che cosa succede? (Scheda 3: Esegui ⟶ Ferma.)

8. Fallo tuo, gli errori e la sfida

Fallo tuo: cambia le frasi nel Memo (per esempio `'Ce l''hai fatta in '`), il titolo della finestra (Caption del form) e il colore del form.
Tieni pronto il codice per la verifica: seleziona dalla riga `var` all'ultimo `end;`, **Ctrl+C**; nella verifica c'è una domanda dove lo incolli.
Errori tipici: manca `Randomize` (escono sempre gli stessi tiri); manca `tiri := tiri + 1` (dice sempre 0 tiri); `while dado = magico` invece di `<>` (non fa nessun giro); numero magico 9 (ciclo infinito).
Sfida: due dadi, finché escono **due 6** insieme (`while (d1 <> 6) or (d2 <> 6) do`). Quanti tiri servono?

Changelog

1. **v1.0 (29/10/2026)**: prima versione, dalle fonti della lezione.